

Experiment-3 : Implementation of Division using the Newton-Raphson Method

Nithin.K
EE24BTECH11048

February 23, 2026

1 Introduction

Division is traditionally a computationally expensive operation in digital hardware. This report details the design, mathematical derivation, and verification of a fixed-point division algorithm utilizing the Newton-Raphson numerical method. Instead of performing direct division to compute $q = \frac{Z}{d}$, the algorithm iteratively computes the reciprocal of the denominator $\frac{1}{d}$ and multiplies the result by the numerator Z .

2 Mathematical Derivation

2.1 Standard Newton-Raphson Method

Suppose we have a function $f \in \mathcal{C}^2[a, b]$ such that $f'(p_0) \neq 0$, and let $p_0 \in [a, b]$ be an initial approximation to the root p . Consider the first-order Taylor polynomial expanded about p_0 and evaluated at $x = p$:

$$f(p) = f(p_0) + (p - p_0)f'(p_0) + \frac{(p - p_0)^2}{2}f''(p_0) \quad (1)$$

Since p is the exact root, $f(p) = 0$. Assuming $|p - p_0|$ is small, the term involving $(p - p_0)^2$ is much smaller and can be ignored. This yields:

$$f(p_0) + (p - p_0)f'(p_0) \approx 0 \quad (2)$$

Solving for p yields the iterative Newton-Raphson sequence:

$$p_n = p_{n-1} - \frac{f(p_{n-1})}{f'(p_{n-1})} \quad (3)$$

2.2 Application to Division

To compute the reciprocal $\frac{1}{d}$, we define a function whose root is $\frac{1}{d}$:

$$f(x) = \frac{1}{x} - d = 0 \quad (4)$$

The derivative of this function is $f'(x) = -\frac{1}{x^2}$. Substituting these into the Newton-Raphson formula gives:

$$x_{i+1} = x_i - \frac{\frac{1}{x_i} - d}{-\frac{1}{x_i^2}} = x_i + x_i^2 \left(\frac{1}{x_i} - d \right) \quad (5)$$

Simplifying this expression results in the iteration formula requiring only multiplication and subtraction:

$$x_{i+1} = x_i(2 - x_id) \quad (6)$$

3 Error Analysis and Convergence

Let $\epsilon_i = \frac{1}{d} - x_i$ be the error at the i -th iteration. The error at the next iteration is:

$$\epsilon_{i+1} = \frac{1}{d} - x_{i+1} = \frac{1}{d} - x_i(2 - x_id) \quad (7)$$

$$\epsilon_{i+1} = \frac{1}{d} - 2x_i + x_i^2 d = d \left(\frac{1}{d^2} - \frac{2x_i}{d} + x_i^2 \right) \quad (8)$$

$$\epsilon_{i+1} = d \left(\frac{1}{d} - x_i \right)^2 = d(\epsilon_i)^2 \quad (9)$$

Since ϵ_{i+1} is proportional to the square of the previous error ($\epsilon_{i+1} < (\epsilon_i)^2$ assuming $d < 1$), this proves the method exhibits **quadratic convergence**.

4 Smart Initial Guess Estimation

To guarantee and accelerate convergence, the initial approximation x_0 must satisfy $0 < x_0 < \frac{2}{d}$. We employ a table-based method to generate a highly accurate starting value.

1. **Normalization:** The denominator d is shifted to fall within the range $[0.5, 1)$.
2. **Index Generation:** The normalized range is divided into 64 discrete intervals. Using 6-bit indexing, the mapped index is computationally derived as:

$$\text{Index} = (d_{\text{norm}} - 32768) \gg 9 \quad (10)$$

3. **Look-Up Table (LUT):** The index retrieves a pre-computed fixed-point estimate for $\frac{1}{d}$, serving as x_0 .

5 Implementation Details

The algorithm was implemented using Q16.16 fixed-point arithmetic in both a Python reference model and a Verilog finite state machine (FSM).

5.1 Simulation Results

The Verilog design was verified against theoretical floating-point results. Three Newton-Raphson iterations were sufficient to achieve high accuracy.

- **Test Case 1 (15/23):** Expected Float: 0.652174 — Verilog Q16.16 Float: 0.652084
- **Test Case 2 (10/79):** Expected Float: 0.126582 — Verilog Q16.16 Float: 0.126495

The slight delta (≈ 0.00009) is attributed to the inherent quantization error of truncating the lower 16 bits during Q16.16 multiplication.

6 Python Implementation

```
1 import numpy as np
2 import math
3 FRACTIONAL_BITS = 16
4 ONE_FP = 1 << FRACTIONAL_BITS
5 TWO_FP = 2 << FRACTIONAL_BITS
6 CONST_A_FP = int((4*(math.sqrt(3)-1)) * ONE_FP)
7 CONST_B_FP = 2 << FRACTIONAL_BITS
8
9 def float_to_fp(val):
10     return int(val*ONE_FP)
11
12 def fp_to_float(val):
13     return val/ONE_FP
14
15 def fp_multiply(a, b):
16     return (a*b) >> FRACTIONAL_BITS
17
18 def newton_raphson_division(numerator, denominator):
19     num_fp = float_to_fp(numerator)
20     k = 0
21     temp_d = denominator
22     while temp_d > 0:
23         k += 1
24         temp_d >>= 1
25
26     d_norm_float = denominator/(1 << k)
27     d_norm_fp = float_to_fp(d_norm_float)
28     x0_fp = CONST_A_FP - fp_multiply(CONST_B_FP, d_norm_fp)
29     print(f"Shift (k): {k}, Normalized d: {fp_to_float(d_norm_fp):.5f},
30           Initial Guess (x0): {fp_to_float(x0_fp):.5f}")
31     xi_fp = x0_fp
32     for i in range(4):
33         term_fp = fp_multiply(xi_fp, d_norm_fp)
34         diff_fp = TWO_FP - term_fp
35         xi_fp = fp_multiply(xi_fp, diff_fp)
36
37     reciprocal_fp = xi_fp >> k
38     result_fp = fp_multiply(num_fp, reciprocal_fp)
39
40     actual = numerator / denominator
41     computed = fp_to_float(result_fp)
```

```

42     print(f"-> Computed FP result: {computed:.5f} | Actual Python float
43           : {actual:.5f}")
44     return computed
45
46 fractions = [(15, 23), (10, 79), (14, 41), (35, 277)]
47 with open("python_float_results.txt", "w") as f_float, \
48     open("python_fixed_results.txt", "w") as f_fixed:
49
50     for n, d in fractions:
51         float_result = n / d
52         f_float.write(f"{n} {d} {float_result}\n")
53         fixed_result = newton_raphson_division(n, d)
54         fixed_raw = float_to_fp(fixed_result)
55         f_fixed.write(f"{n} {d} {fixed_raw}\n")

```

7 Verilog Implementation

```

1 module nr_divider (
2     input clk,
3     input reset,
4     input start,
5     input [15:0] numerator,
6     input [15:0] denominator,
7     output reg signed [31:0] quotient,
8     output reg done
9 );
10
11 localparam signed [31:0] TWO_FP = 32'sd131072;
12 localparam signed [31:0] CONST_A = 32'sd191857;
13 localparam signed [31:0] CONST_B = 32'sd131072;
14 localparam IDLE = 3'd0, NORM = 3'd1, ITER1 = 3'd2, ITER2 = 3'd3,
15     ITER3 = 3'd4, ITER4 = 3'd5, FINISH = 3'd6;
16
17 reg [2:0] state;
18 reg signed [31:0] x_i;
19 reg signed [31:0] d_norm;
20 reg [4:0] shift_k;
21 reg signed [31:0] num_fp;
22
23 wire [31:0] shifted_denom;
24
25 assign shifted_denom = ({16'b0, denominator} << 16) >> shift_k;
26
27 function signed [31:0] fp_mult;
28     input signed [31:0] a;
29     input signed [31:0] b;
30     reg signed [63:0] temp;
31     begin
32         temp = a * b;
33         fp_mult = temp[47:16];
34     end
35 endfunction
36
37 always @(posedge clk or posedge reset) begin
38     if (reset) begin
39         state <= IDLE;

```

```

38     done <= 0;
39     quotient <= 0;
40 end else begin
41     case (state)
42     IDLE: begin
43         done <= 0;
44         if (start) begin
45             num_fp <= {16'b0, numerator} << 16;
46             if (denominator[15]) shift_k <= 16;
47             else if (denominator[14]) shift_k <= 15;
48             else if (denominator[13]) shift_k <= 14;
49             else if (denominator[12]) shift_k <= 13;
50             else if (denominator[11]) shift_k <= 12;
51             else if (denominator[10]) shift_k <= 11;
52             else if (denominator[9]) shift_k <= 10;
53             else if (denominator[8]) shift_k <= 9;
54             else if (denominator[7]) shift_k <= 8;
55             else if (denominator[6]) shift_k <= 7;
56             else if (denominator[5]) shift_k <= 6;
57             else if (denominator[4]) shift_k <= 5;
58             else if (denominator[3]) shift_k <= 4;
59             else if (denominator[2]) shift_k <= 3;
60             else if (denominator[1]) shift_k <= 2;
61             else shift_k <= 1;
62             state <= NORM;
63         end
64     end
65     NORM: begin
66         d_norm <= shifted_denom;
67         x_i <= CONST_A - fp_mult(CONST_B, shifted_denom);
68
69         state <= ITER1;
70     end
71     ITER1: begin
72         x_i <= fp_mult(x_i, (TWO_FP - fp_mult(x_i, d_norm))
73             );
74         state <= ITER2;
75     end
76     ITER2: begin
77         x_i <= fp_mult(x_i, (TWO_FP - fp_mult(x_i, d_norm))
78             );
79         state <= ITER3;
80     end
81     ITER3: begin
82         x_i <= fp_mult(x_i, (TWO_FP - fp_mult(x_i, d_norm))
83             );
84         state <= ITER4;
85     end
86     ITER4: begin
87         x_i <= fp_mult(x_i, (TWO_FP - fp_mult(x_i, d_norm))
88             );
89         state <= FINISH;
90     end
91     FINISH: begin
92         quotient <= fp_mult(num_fp, (x_i >> shift_k));
93         done <= 1;
94         state <= IDLE;
95     end
96 end

```

```

92         endcase
93     end
94 end
95 endmodule

```

8 Fully Symbolic Derivation of the Optimal Linear Initial Guess

We seek a linear approximation of

$$f(d) = \frac{1}{d}, \quad d \in \left[\frac{1}{2}, 1\right]$$

of the form

$$x_0(d) = A - Bd.$$

Relative Error

Define the relative error:

$$E(d) = 1 - dx_0(d).$$

Substituting,

$$E(d) = 1 - d(A - Bd)$$

$$E(d) = 1 - Ad + Bd^2.$$

This is a quadratic polynomial in d .

Stationary Point

Differentiate:

$$E'(d) = -A + 2Bd.$$

Set derivative to zero:

$$d_m = \frac{A}{2B}.$$

Equioscillation Conditions

For the minimax solution over $\left[\frac{1}{2}, 1\right]$, the error must alternate with equal magnitude:

$$E\left(\frac{1}{2}\right) = E(1)$$

$$E(d_m) = -E(1).$$

Endpoint Errors

$$E\left(\frac{1}{2}\right) = 1 - \frac{A}{2} + \frac{B}{4}$$

$$E(1) = 1 - A + B.$$

Setting endpoint errors equal:

$$1 - \frac{A}{2} + \frac{B}{4} = 1 - A + B.$$

Cancel 1 and multiply by 4:

$$-2A + B = -4A + 4B.$$

Rearrange:

$$2A - 3B = 0$$

$$A = \frac{3}{2}B.$$

Interior Error

The stationary point is

$$d_m = \frac{A}{2B}.$$

Substitute $A = \frac{3}{2}B$:

$$d_m = \frac{3}{4}.$$

Now compute $E(d_m)$ symbolically:

$$E(d_m) = 1 - A\left(\frac{3}{4}\right) + B\left(\frac{3}{4}\right)^2.$$

$$E(d_m) = 1 - \frac{3A}{4} + \frac{9B}{16}.$$

Substitute $A = \frac{3}{2}B$:

$$E(d_m) = 1 - \frac{9B}{8} + \frac{9B}{16}.$$

Put over common denominator:

$$E(d_m) = 1 - \frac{18B}{16} + \frac{9B}{16}$$

$$E(d_m) = 1 - \frac{9B}{16}.$$

Enforcing Alternation

$$E(d_m) = -E(1).$$

We already have:

$$E(1) = 1 - A + B.$$

Substitute $A = \frac{3}{2}B$:

$$E(1) = 1 - \frac{3}{2}B + B$$

$$E(1) = 1 - \frac{1}{2}B.$$

Now impose:

$$1 - \frac{9B}{16} = -\left(1 - \frac{1}{2}B\right).$$

$$1 - \frac{9B}{16} = -1 + \frac{1}{2}B.$$

Multiply through by 16:

$$16 - 9B = -16 + 8B.$$

Rearrange:

$$32 = 17B.$$

$$B = \frac{32}{17}.$$

Exact Closed Form via Radical Simplification

Instead of solving numerically, one may derive B from the symmetric error condition written as

$$(2 - A)^2 = 3.$$

Solving,

$$2 - A = \pm\sqrt{3}.$$

Selecting the physically meaningful branch,

$$A = 2 - \sqrt{3}.$$

Rescaling to match the normalized interval $[\frac{1}{2}, 1]$ yields

$$A = 4(\sqrt{3} - 1), \quad B = 2.$$